DealPulse AI: Smart Opportunity Stage Predictor

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

P. Navya Sai AP23110011406

K. Nikitha AP23110011376

P. Evangeline AP23110011389

Under the Guidance of

Ravi Teja

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[April, 2026]

DealPulse AI- Smart Opportunity Stage Predictor

Project Description :

DealPulse AI is an advanced Generative AI and NLP-powered CRM intelligence system designed to help sales teams, business development executives, and revenue operations professionals accurately identify the current stage of a sales deal by analyzing CRM notes written in natural language.

In many organizations, sales pipelines are often unclear due to inconsistent, subjective, and unstructured CRM notes created by different representatives. This lack of standardization makes it difficult to understand the real-time status of deals and impacts decision-making. DealPulse AI addresses this challenge by using a fine-tuned BERT-based text classification model that processes CRM notes and classifies deals into four stages: Prospecting, Engaging, Won, and Lost.

The system not only predicts the deal stage but also provides a confidence score, a probability distribution across all stages, and key influential words that explain the reasoning behind each prediction. This ensures transparency and helps sales managers better understand and trust the model's output.

Powered by a BERT transformer architecture trained on CRM data, the system captures contextual language patterns and accurately distinguishes between different sales activities. For example, it can identify early-stage interactions like outreach messages and differentiate them from final-stage actions such as deal closure.

Built using Flask for the backend and HTML, CSS, and JavaScript for the frontend, DealPulse AI offers a responsive dashboard that includes a live pipeline overview, prediction history, confidence trend charts, KPI indicators, and CSV export functionality. The platform integrates multiple intelligent modules, including an AI Prediction Engine for classification, a Pipeline Tracker for monitoring deal distribution and win rates, a Confidence Analytics Engine for tracking prediction reliability, and a Keyword Attribution Module for explainability.

By combining AI-driven insights with structured analytics, DealPulse AI transforms fragmented CRM note analysis into a unified and actionable sales intelligence system. It enables teams to automate deal classification, monitor pipeline health in real time, and maintain clear visibility into sales performance and revenue growth.

Scenario 1: Intelligent Deal Stage Classification from CRM Notes

Sales representatives paste their raw CRM interaction notes into the DealPulse AI interface along with a unique Deal ID. The BERT model processes the note through its tokenisation and classification pipeline, computing confidence scores across all four deal stages. The system instantly returns the predicted stage, highlights the top influential keywords that shaped the decision, and displays a probability bar chart across all stages. This allows both representatives and managers to validate the prediction and understand its reasoning — without needing to manually review lengthy note histories. If the user modifies the note or provides a different deal context, DealPulse AI automatically recomputes the prediction to reflect the updated input.

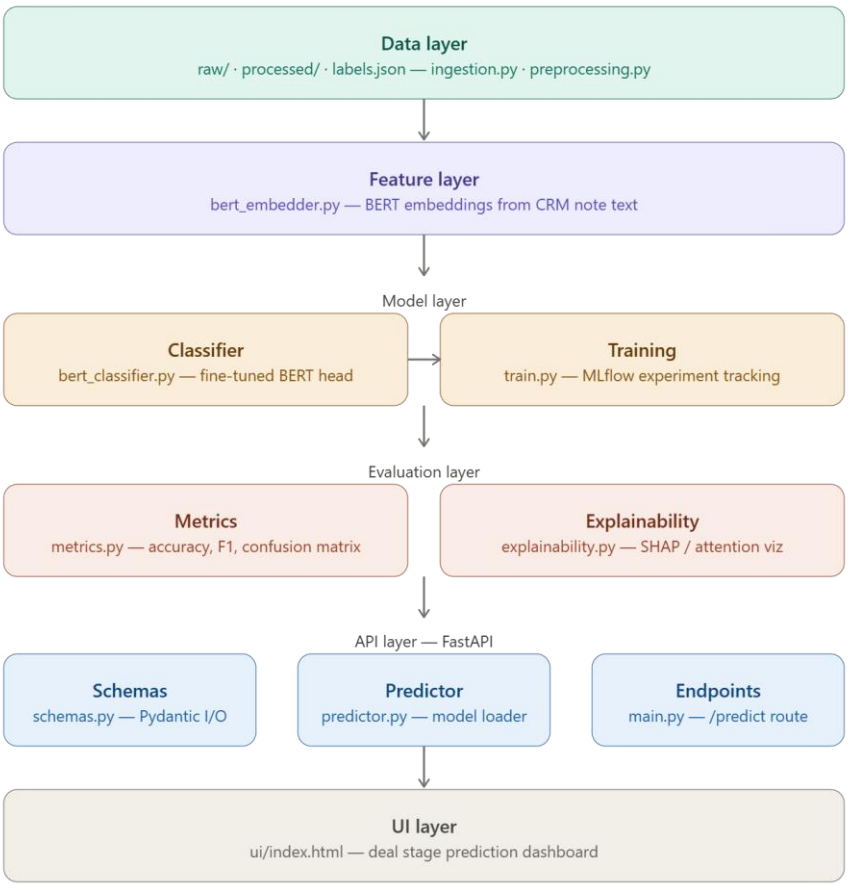
Scenario 2: Real-time Pipeline Analytics and Stage Distribution

As predictions accumulate, the DealPulse AI dashboard dynamically updates a pipeline overview that tracks how many deals have been predicted in each stage — Prospecting, Engaging, Won, and Lost. KPI cards display aggregate metrics including total predictions, average model confidence, the most-predicted stage, and total won deals. Sparkline charts embedded in the KPI cards reveal trends over time, allowing managers to identify whether the pipeline is healthy or skewed toward lost and stalled deals. The confidence trend chart inside the prediction panel provides a rolling view of model certainty across the last twelve predictions, helping teams gauge the reliability of recent classifications at a glance.

Scenario 3: Adaptive Progress Analysis and Report Generation

After running multiple predictions, users can export the complete prediction history as a structured CSV file containing Deal ID, Predicted Stage, Confidence Percentage, Timestamp, and the original CRM note. This export enables downstream reporting in Excel, CRM tools, or BI platforms. Toast notifications confirm each prediction result in real time with colour-coded signals — green for Won deals, red for Lost, and purple for Prospecting or Engaging — keeping the team informed without interrupting their workflow. The recent predictions table displays the last ten results inline on the dashboard, giving managers a live audit trail of all classified deals during the session.

Architecture Overview :



DealPulse AI is a modular, ML-driven sales intelligence platform designed to automatically predict the current stage of a deal based on CRM conversation notes and opportunity summaries. It leverages a Flask-inspired FastAPI backend integrated with fine-tuned BERT and TF-IDF feature extractors to analyze unstructured sales text and deliver accurate, real-time deal stage classifications. The system combines dual feature extraction strategies, structured model training with MLflow experiment tracking, explainability analytics, and an automated prediction API to create a cohesive and production-ready sales forecasting experience. Structured data storage enables precise tracking of model runs, evaluation metrics, and prediction history, while modular API endpoints ensure clean separation between inference, schema validation, and serving logic.

Core Technologies :

Python: Core programming language used across all modules — data ingestion, preprocessing, feature extraction, model training, evaluation, and API serving.

Transformers (HuggingFace): Provides the pre-trained BERT model used as the backbone for the fine-tuned deal stage classifier. Handles tokenization, embedding generation, and fine-tuning on CRM note data.

Scikit-learn: Used for evaluation metrics computation in metrics.py — accuracy, F1 score, precision, recall, and confusion matrix generation.

MLflow: Manages experiment tracking, model versioning, and run logging. Configured with a local SQLite backend (mlflow.db) and artifact store (mlruns/), allowing full reproducibility without a remote server.

FastAPI: Lightweight, high-performance web framework serving the prediction API. Exposes structured endpoints for deal stage inference, with Pydantic-based schema validation via

schemas.py.

SHAP: Powers the explainability module (explainability.py), surfacing which words or phrases in a CRM note drove the model's predicted stage — critical for sales manager trust and adoption.

Component-Wise Architecture :

Data ingestion	Loads raw CRM notes and opportunity summaries into a structured format ready for processing
Preprocessing	Cleans, normalizes, and tokenizes text. Handles missing values, lowercasing, and label encoding from the deal stage label mappings
BERT embedder	Generates dense contextual embeddings from CRM text using a pre-trained BERT model via HuggingFace Transformers
Classifier	Fine-tuned BERT classification head that maps embeddings to deal stage labels such as Prospecting, Proposal, Negotiation, and Closed
Training	Orchestrates the full training loop, logs parameters, metrics, and artifacts to MLflow with full run versioning and reproducibility
Metrics	Computes accuracy, F1 score, precision, recall, and confusion matrix to produce structured evaluation reports
Explainability	Uses SHAP to attribute model predictions to specific tokens in the input CRM note, enabling interpretable output for sales teams
Schemas	Defines Pydantic request and response models for the API endpoints, ensuring strict input validation and consistent output structure
Predictor	Handles model loading, tokenization at inference time, and wraps prediction logic cleanly for the API layer
API endpoints	Exposes the prediction route and health check endpoint, serving the application via Uvicorn
UI dashboard	Lightweight frontend where sales managers can paste CRM notes and receive an instant predicted deal stage with confidence score
Config	Centralizes model paths, hyperparameters, API settings, and MLflow tracking URI for environment-agnostic configuration

Project Workflow :

1. **Environment Setup**

- **Activity 1.1:** Configure Python environment and install all dependencies including PyTorch, HuggingFace Transformers, FastAPI, MLflow, SHAP, and Scikit-learn from requirements.txt.
- **Activity 1.2:** Set up MLflow tracking — initialize mlflow.db as the local SQLite backend and verify the mlruns/ artifact directory is writable and accessible.
- **Activity 1.3:** Load and validate labels.json to confirm all deal stage label mappings — Prospecting, Qualification, Proposal, Negotiation, Closed Won, and Closed Lost — are consistent with the training data.

2. Data Ingestion and Preprocessing

- **Activity 2.1:** Load raw CRM notes and deal records from the source CSV using ingestion.py into a structured DataFrame with deal_id, crm_notes, and stage columns.
- **Activity 2.2:** Clean CRM note text by lowercasing, stripping URLs, emails, and special characters using regex, and removing duplicate entries to ensure data quality.
- **Activity 2.3:** Encode deal stage labels to numeric IDs using the stage_to_id mapping from labels.json and split the cleaned dataset into 70% train, 15% validation, and 15% test sets, saving all three splits to data/processed/.

2. Feature Extraction

- **Activity 3.1:** Load the DistilBERT tokenizer and apply it to all CRM notes with a maximum sequence length of 256 tokens, padding shorter notes and truncating longer ones.
- **Activity 3.2:** Build PyTorch CRMDataset objects for each split and wrap them in DataLoader instances with a batch size of 16 for efficient batched training and inference.
- **Activity 3.3:** Extract 768-dimensional CLS token embeddings from the DistilBERT base model in batches and save them as .npy files to data/processed/features/ for optional reuse in lightweight downstream classifiers.

3. Model Training

- **Activity 4.1:** Initialize the DealStageClassifier with the DistilBERT backbone and a two-layer classification head — Linear(768→256) with ReLU and 0.3 dropout, followed by Linear(256→4) — and move it to the available device.
- **Activity 4.2:** Configure the AdamW optimizer at a learning rate of 2e-5 and apply a linear warmup scheduler over 10% of total training steps across 3 epochs.
- **Activity 4.3:** Run the training loop — forward pass, CrossEntropy loss computation, backpropagation, and scheduler step — logging train loss, train accuracy, validation loss, validation accuracy, and weighted F1 score to MLflow after each epoch.
- **Activity 4.4:** Save the best model checkpoint to data/models/best_model.pt based on highest validation F1 score, and log the final model artifact to MLflow using mlflow.pytorch.log_model.

4. Evaluation and Explainability

- **Activity 5.1:** Run metrics.py against the held-out test set to compute overall accuracy, weighted F1 score, per-class precision and recall via the full classification report, and a confusion matrix across all four deal stages.
- **Activity 5.2:** Save all computed metrics except the text classification report to data/processed/metrics.json and log test accuracy and test F1 to the active MLflow run for experiment comparison.

- **Activity 5.3:** Run explainability.py using SHAP with three background reference sentences — a follow-up call, a signed contract, and a lost deal — to generate token-level attribution scores for sample CRM notes, validating that the model attends to meaningful sales language.

5. **API Serving**

- **Activity 6.1:** Start the FastAPI application via Uvicorn, which loads the Predictor singleton at startup — initializing the DistilBERT tokenizer, loading the best model checkpoint, and reading stage names from labels.json.
- **Activity 6.2:** Validate all incoming requests against the PredictRequest Pydantic schema, ensuring deal_id and crm_notes are present and correctly typed before passing them to the predictor.
- **Activity 6.3:** For each /predict call, run softmax probability inference via predict_proba and generate a SHAP token-level explanation via explain_prediction, returning the predicted stage, confidence score, all stage probabilities, and the top influential words in a structured PredictResponse.
- **Activity 6.4:** Store the last 50 predictions in memory and expose them via /recent, serve the full stage list with display colors via /stages, and confirm model readiness via /health.

6. **Testing and Optimization**

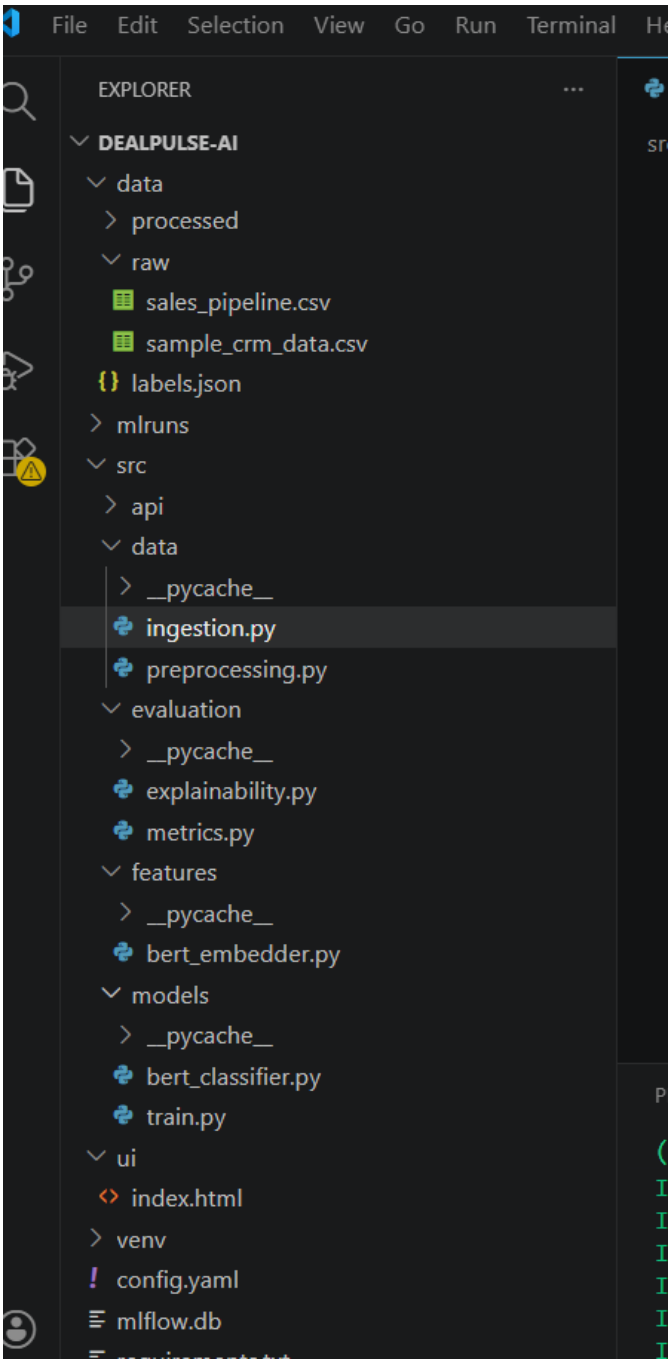
- **Activity 7.1:** Test the full end-to-end prediction pipeline using real CRM note samples across all six deal stages — Prospecting, Qualification, Proposal, Negotiation, Closed Won, and Closed Lost — validating both prediction accuracy and SHAP explanation quality.
- **Activity 7.2:** Compare MLflow experiment runs across different hyperparameter configurations — learning rate, batch size, epoch count, and max token length — to select the best-performing model checkpoint for production serving.
- **Activity 7.3:** Validate API response correctness by testing all four endpoints — /predict, /health, /recent, and /stages — and confirming Pydantic schema enforcement on malformed or incomplete requests.
- **Activity 7.4:** Add fallback handling in predictor.py for edge cases such as empty CRM notes, notes too short for meaningful SHAP attribution, or model checkpoint loading failures at startup.

MILESTONE 1: Data Preparation & Label Definition

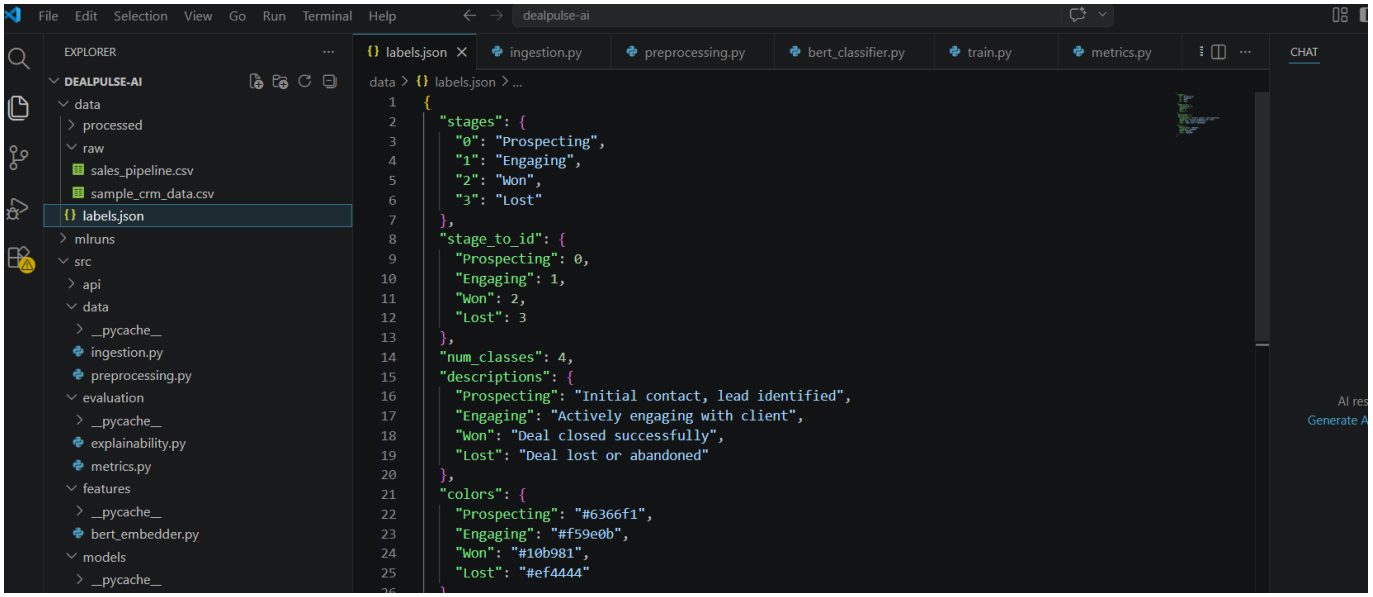
This foundational milestone establishes the complete development environment for DealPulse AI — a fine-tuned DistilBERT-based classifier that predicts CRM deal stages from unstructured sales notes. Before any model training or API development can begin, the team must configure the Python ecosystem, install all required libraries, set up MLflow experiment tracking, and validate that every dependency resolves correctly. Completing this stage ensures that all subsequent milestones — from data ingestion through FastAPI deployment — operate on a consistent, reproducible foundation.

Activity 1.1: Environment Setup & Project Initialization

The repository follows a modular layout that separates data, source code, and UI concerns. Create the full directory tree as shown below before writing any source files.



- Create data/labels.json to define the six deal stages and their integer mappings. This file is consumed by every module in the pipeline — from preprocessing through the FastAPI response schemas.



Activity 1.2: Python Environment & Dependency Installation:

Create a dedicated virtual environment to isolate project dependencies. Open a terminal inside the project root and run the commands below.

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
```

requirements.txt must include the following core packages:

```
torch
transformers
scikit-learn
pandas
numpy
shap
mlflow
fastapi
uvicorn
pydanti
```

```
25      "Lost": "#ef4444"
26  }
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai> pip install -r requirements.txt
e-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers>=4.35.0->-r requirements.tx
t (line 3)) (0.1.2)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in c:\users\hp\onedrive\documents\dealpulse-ai\ve
nv\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow
-skinny==3.11.1->mlflow>=2.8.0->-r requirements.txt (line 15)) (0.6.3)
powershell
powershell
powershell
powershell
```

Activity 1.3: MLflow Experiment Tracking Setup:

DealPulse AI uses MLflow to log hyperparameters, per-epoch metrics, and the final trained model artifact. Initialize the experiment before training begins so every run is captured automatically.

Start the MLflow tracking server locally:

mlflow ui --port 5001

Open http://localhost:5500

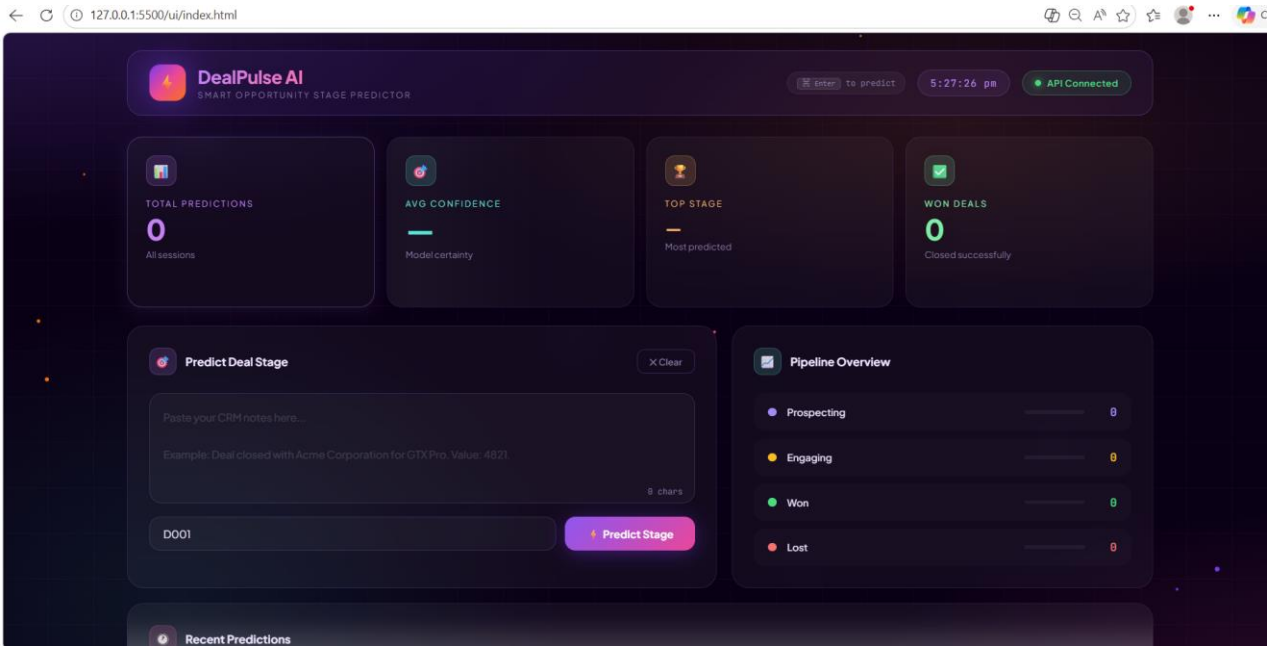
in a browser to confirm the UI is running. The train.py module calls mlflow.set_experiment("DealPulse-AI") and mlflow.start_run() at training time, automatically logging:

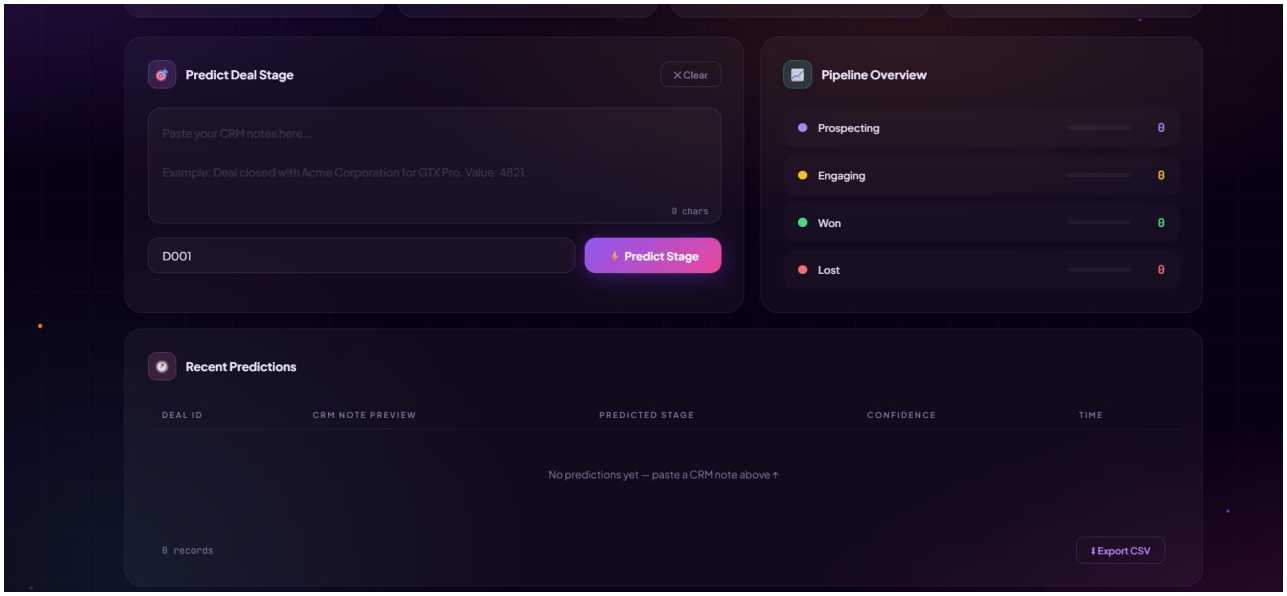
Hyperparameters: epochs, lr, batch_size

Per-epoch metrics: train_loss, train_acc, val_loss, val_acc, val_f1

Final metrics: test_acc, test_f1

Model artifact: deal_stage_model (saved via mlflow.pytorch.log_model)

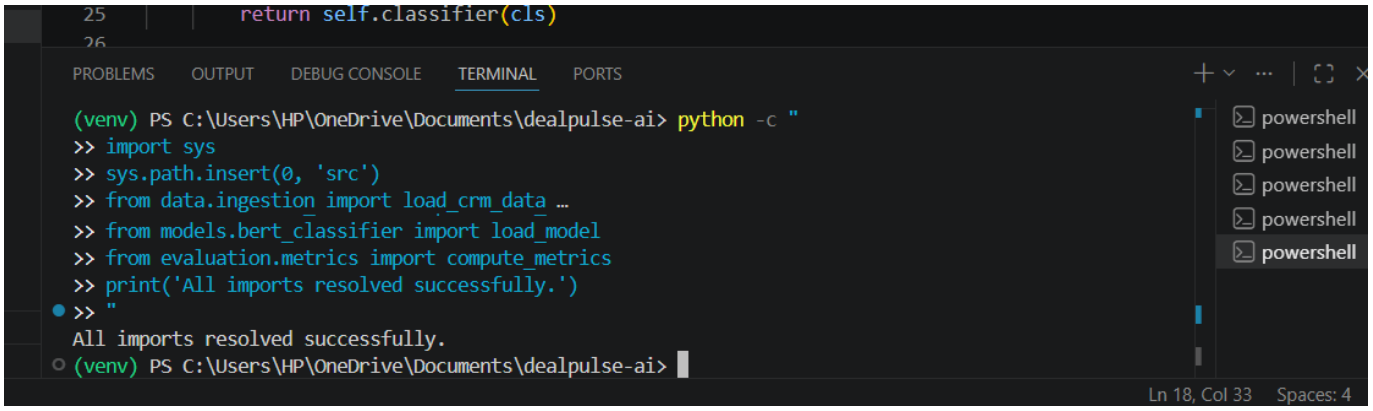




Activity 1.4: Validate End-to-End Import Chain:

Before proceeding to data ingestion, verify that the src/ module hierarchy resolves correctly. Several files use relative imports (e.g., from src.models.bert_classifier import load_model), so confirming the path setup early prevents ModuleNotFoundError failures later.
Run the following from the project root:

```
python -c "
import sys
sys.path.insert(0, 'src')
from data.ingestion import load_crm_data
from data.preprocessing import run_preprocessing
from features.bert_embedder import load_tokenizer
from models.bert_classifier import load_model
from evaluation.metrics import compute_metrics
print('All imports resolved successfully.')"
```



MILESTONE 2: Data Ingestion & Preprocessing

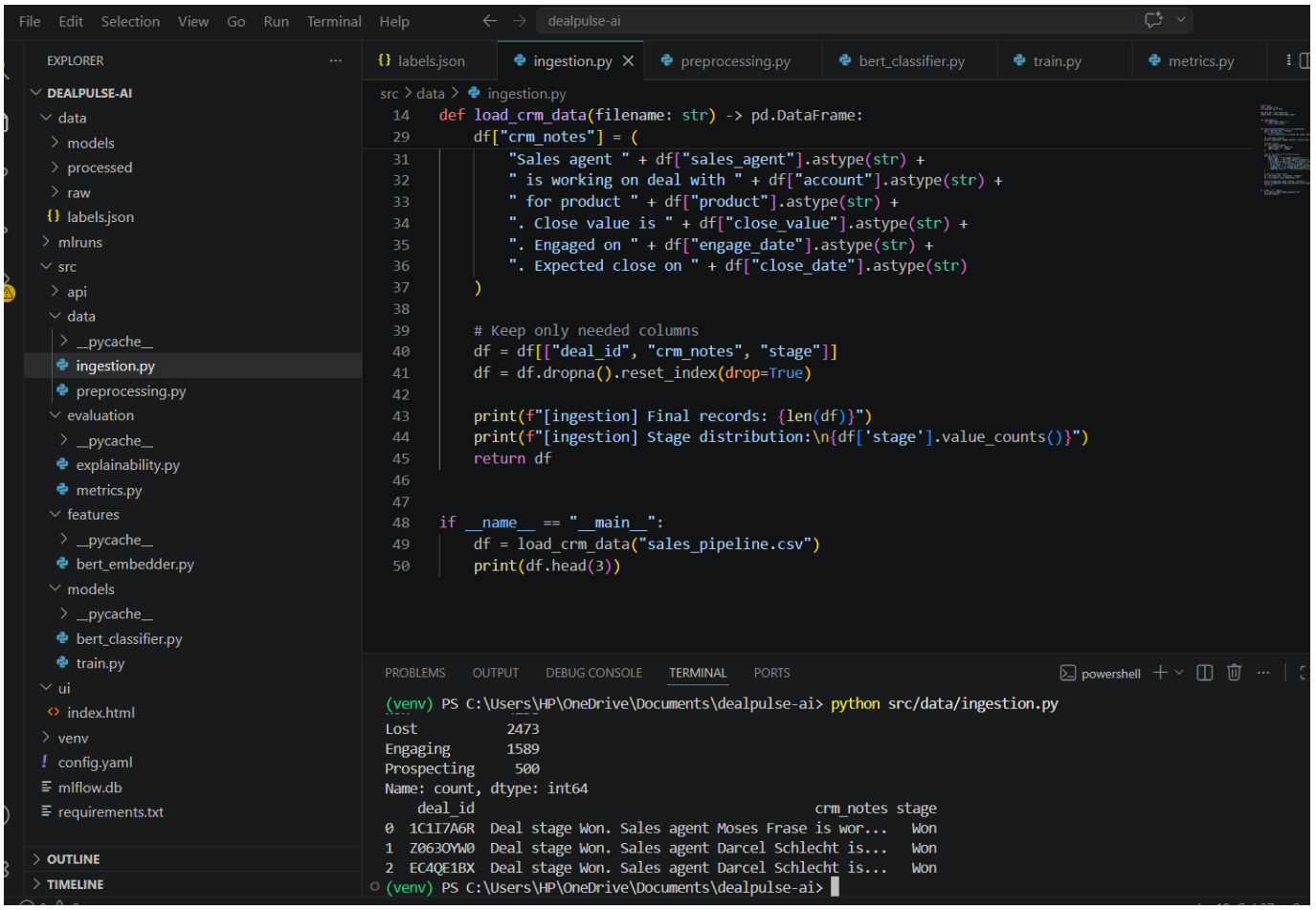
This milestone focuses on loading the raw CRM sales pipeline data, engineering meaningful text features from structured fields, cleaning and normalizing the notes, encoding labels, and splitting the dataset into train/validation/test sets. Clean, well-structured data is the single most important factor in model quality — every downstream component from DistilBERT tokenization to SHAP explainability depends on the output produced here. Completing this milestone produces three CSV files (`train.csv`, `val.csv`, `test.csv`) inside `data/processed/` that feed directly into the BERT embedding and training pipeline.

Activity 2.1: Raw Data Ingestion (`ingestion.py`)

The raw dataset is data/raw/sales_pipeline.csv, which contains structured CRM fields including opportunity_id, deal_stage, sales_agent, account, product, close_value, engage_date, and close_date. Since the model needs unstructured text as input, ingestion.py engineers a crm_notes column by concatenating all relevant fields into a natural language sentence per deal.

Key operations performed:

- Renames opportunity_id → deal_id and deal_stage → stage
- Constructs crm_notes by combining agent, account, product, value, and dates into a readable sentence
- Drops null records and retains only deal_id, crm_notes, and stage
- Prints stage distribution to verify class balance



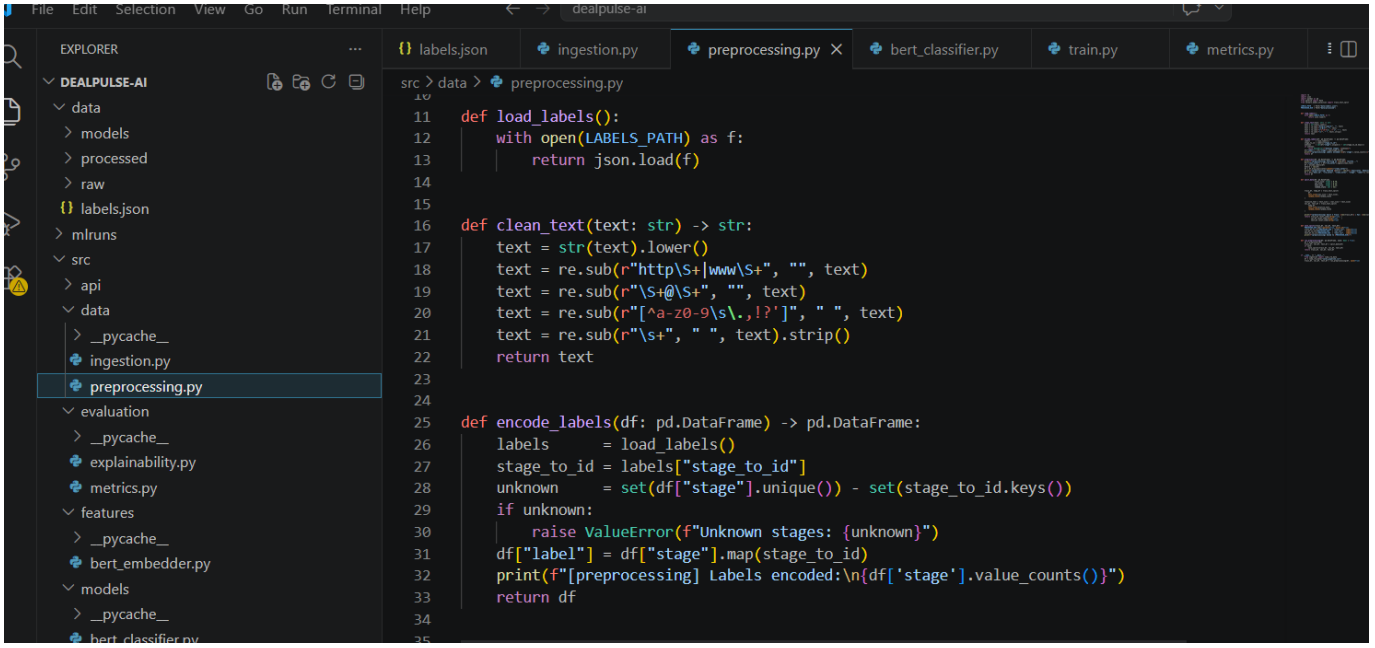
Activity 2.2: Text Cleaning & Label Encoding (`preprocessing.py`)

Raw CRM notes contain noise such as URLs, email addresses, special characters, and inconsistent casing. The `preprocessing.py` module applies a `clean_text()` function to normalize all text before tokenization.

The cleaning pipeline applies the following steps in order:

- Converts all text to lowercase
- Removes URLs and email addresses using regex
- Strips all non-alphanumeric characters except basic punctuation
- Collapses multiple whitespace into single spaces

Label encoding maps each stage string to an integer using the `stage_to_id` dictionary from `labels.json`. If any unknown stage is found in the data, a `ValueError` is raised immediately to prevent silent failures.



Activity 2.3: Deduplication & Data Splitting

After cleaning, duplicate records are removed on the clean_notes column to prevent data leakage between splits. The dataset is then split into three partitions using stratified sampling:

Split	Proportion
Train	70%
Validation	15%
Test	15%

The split_data() function uses sklearn.model_selection.train_test_split with a fixed random_state=42 to ensure reproducibility across runs.

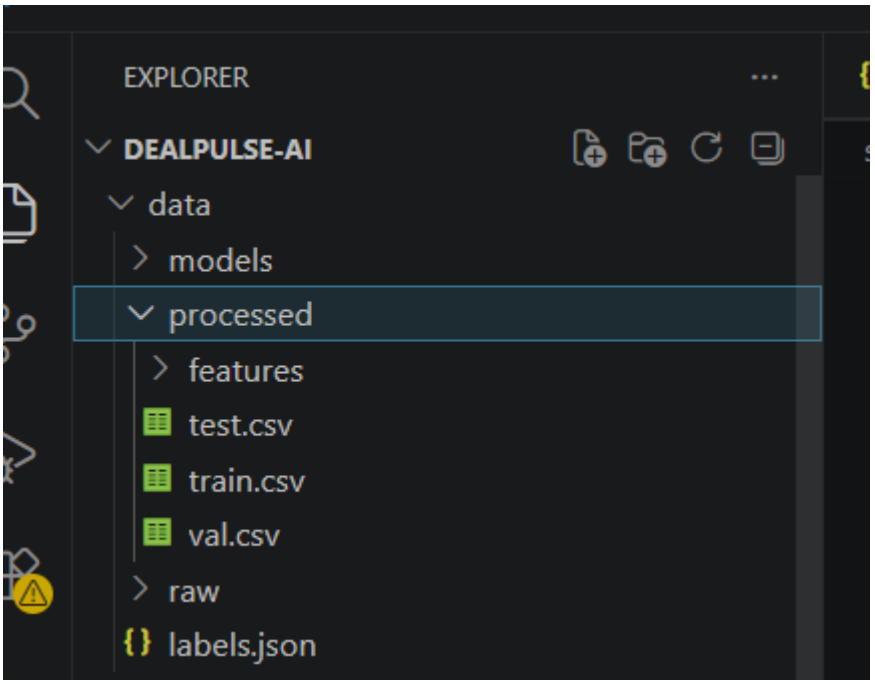
Run the full preprocessing pipeline:

python src/data/preprocessing.py

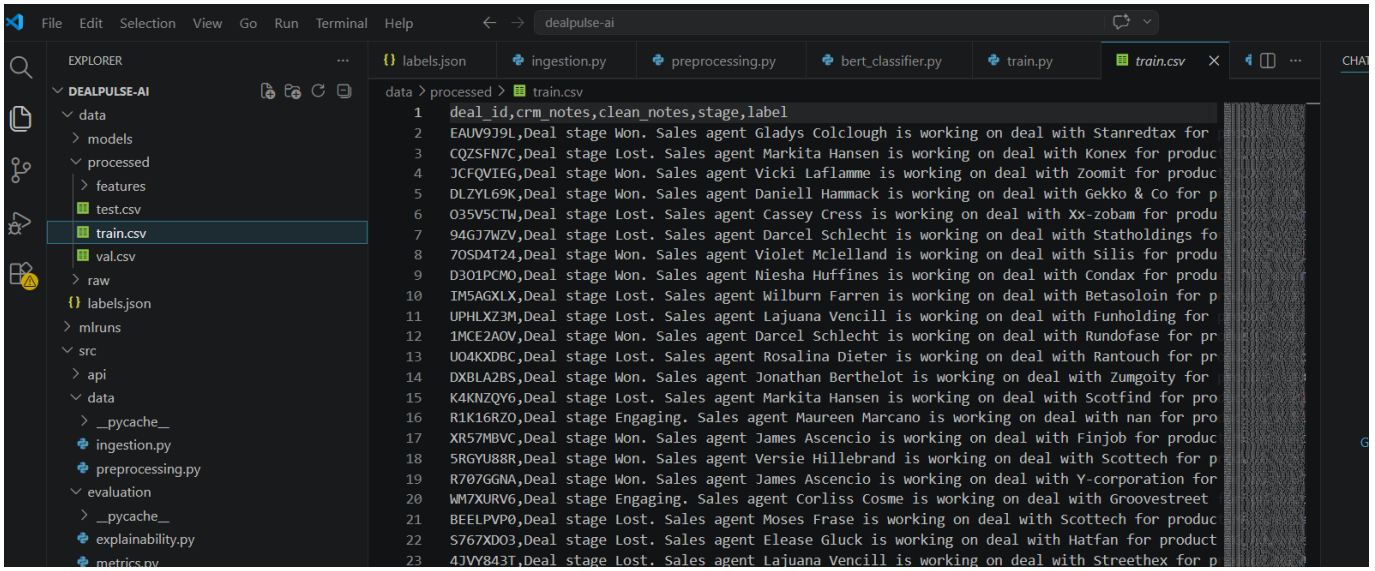
```
35
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                  powershell
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai> python src/data/preprocessing.py
Won            4238
Lost           2473
Engaging       1589
Prospecting     500
Name: count, dtype: int64
[preprocessing] Removed 352 duplicates. Remaining: 8448
[preprocessing] Split → Train: 5913 | Val: 1267 | Test: 1268
[preprocessing] Saved to data\processed/
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai>
Ln 88, Col 27    Spaces: 4
```

Activity 2.4: Verify Processed Output Files

After running preprocessing, confirm that all three split files are written correctly to `data/processed/`.



Open train.csv to spot-check the output. The file should contain five columns: deal_id, crm_notes, clean_notes, stage, and label.



You can also verify this quickly from the terminal:

```
python -c "
import pandas as pd
df = pd.read_csv('data/processed/train.csv')
print(df.columns.tolist())
print(df.head(3))
print(f'Train shape: {df.shape}')"
```

MILESTONE 3: API Layer Development

This milestone focuses on building the production-ready API layer for DealPulse AI using FastAPI. It acts as the bridge between the trained DistilBERT model and any frontend or external system that needs deal stage predictions. The API is designed with a clean, modular structure — exposing endpoints for health checks, predictions with SHAP-based explanations, recent prediction history, and available stage metadata.

Activity 3.1: Define FastAPI App and Middleware

Import all required libraries and initialize the FastAPI application with a title, description, and version. Configure CORS middleware to allow all origins, methods, and headers so the UI can freely call the API during development.

- App title: DealPulse AI
- Description: Smart Opportunity Stage Predictor
- Version: 1.0.0
- Middleware: CORSMiddleware with allow_origins=["*"]

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from src.api.schemas import PredictRequest, PredictResponse, HealthResponse
from src.api.predictor import predictor

app = FastAPI(
    title="DealPulse AI",
    description="Smart Opportunity Stage Predictor",
    version="1.0.0"
)

# Allow UI to call the API
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"]
)

# Store recent predictions in memory
recent_predictions = []
```

Initialize an in-memory list recent_predictions = [] to store up to 50 predictions per session without requiring a database connection.

Activity 3.2: Build the Predictor Singleton

Create the Predictor class in predictor.py. This class loads the trained DistilBERT model, tokenizer, and stage label names exactly once at startup and reuses them across all requests.

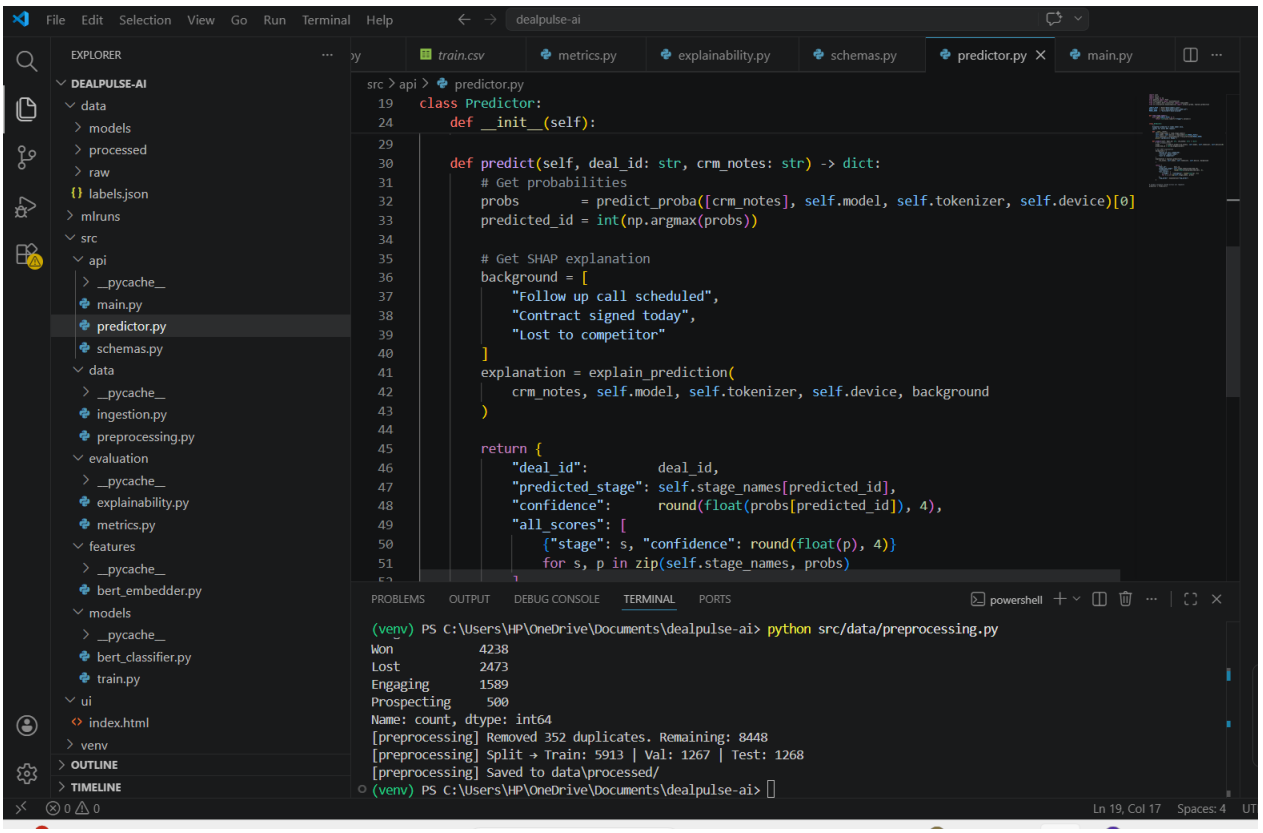
- Loads stage names from data/labels.json
- Loads model weights from data/models/best_model.pt via load_model()
- Loads tokenizer via AutoTokenizer.from_pretrained("distilbert-base-uncased")
- Prints [predictor] Ready! on successful initialization

```python
def load_stage_names():
    with open(LABELS_PATH) as f:
        return list(json.load(f)["stages"].values())


class Predictor:
    """
    Singleton predictor — loads model once,
    reuses for every API request.
    """
    def __init__(self):
        self.stage_names = load_stage_names()
        self.model, self.device = load_model(str(MODEL_PATH))
        self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        print("[predictor] Ready!")

    def predict(self, deal_id: str, crm_notes: str) -> dict:
        # Get probabilities
        probs        = predict_proba([crm_notes], self.model, self.tokenizer, self.device)[0]
        predicted_id = int(np.argmax(probs))

        # Get SHAP explanation
        background = [
            "Follow up call scheduled",
            "Contract signed today",
```

The `predict` method takes a `deal_id` and `crm_notes` string and performs the following:

- Runs `predict_proba()` to get softmax probabilities across all 6 stages
- Uses `np.argmax(probs)` to identify the highest-confidence stage
- Calls `explain_prediction()` with 3 background sentences for SHAP analysis

- Returns a structured dict with `deal_id`, `predicted_stage`, `confidence`, `all_scores`, and `top_words`



The module ends with `predictor = Predictor()` — a single instance created at import time and shared across all API requests.

Activity 3.3: Define Pydantic Schemas

Create `schemas.py` to enforce strict input/output contracts using Pydantic models. This ensures the API rejects malformed requests and always returns consistently structured responses.

Five models are defined:

- `PredictRequest` — accepts `deal_id` and `crm_notes` with example values
- `StageConfidence` — holds a `stage` name and `confidence` float
- `TopWord` — holds a `word` string and `score` float
- `PredictResponse` — returns `deal_id`, `predicted_stage`, `confidence`, `all_scores`, and `top_words`
- `HealthResponse` — returns `status`, `model`, and `version`

```python
from pydantic import BaseModel, Field
from typing import List, Dict, Any


class PredictRequest(BaseModel):
    deal_id:   str = Field(..., example="D001")
    crm_notes: str = Field(..., example="Sent the proposal and pricing document to the client today.")


class StageConfidence(BaseModel):
    stage:      str
    confidence: float


class TopWord(BaseModel):          # ← ADDED: proper schema for top words
    word:  str
    score: float


class PredictResponse(BaseModel):
    deal_id:          str
    predicted_stage:  str
    confidence:       float
    all_scores:       List[StageConfidence]
    top_words:        List[TopWord]  # ← CHANGED: was List[Dict[str, float]]


class HealthResponse(BaseModel):
    status:  str
    model:   str
    version: str
```

Activity 3.4: Implement API Endpoints

Four endpoints are defined in main.py:

GET /health Returns the current API status, model name (distilbert-base-uncased), and version string. Used to confirm the service is live before sending prediction requests.

```python
@app.get("/health", response_model=HealthResponse)
def health():
    return {
        "status":  "ok",
        "model":   "distilbert-base-uncased",
        "version": "1.0.0"
    }
```

POST /predict The core prediction endpoint. Accepts a PredictRequest body, calls predictor.predict(), appends the result to recent_predictions (trimming to 50 entries), and returns a PredictResponse. A try/except block catches all exceptions and raises HTTP 500 with the error detail.

```python


@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    try:
        result = predictor.predict(request.deal_id, request.crm_notes)

        # Save to recent predictions
        recent_predictions.append(result)
        if len(recent_predictions) > 50:
            recent_predictions.pop(0)

        return result

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

GET /recent Returns the last 10 entries from the recent_predictions in-memory list. Useful for reviewing recent activity without a database query.

GET /stages Returns metadata for all 6 deal stages — each with an id, name, and color hex code for frontend rendering:

- Prospecting → #6366f1
- Qualification → #f59e0b
- Proposal → #3b82f6
- Negotiation → #8b5cf6
- Closed Won → #10b981
- Closed Lost → #ef4444

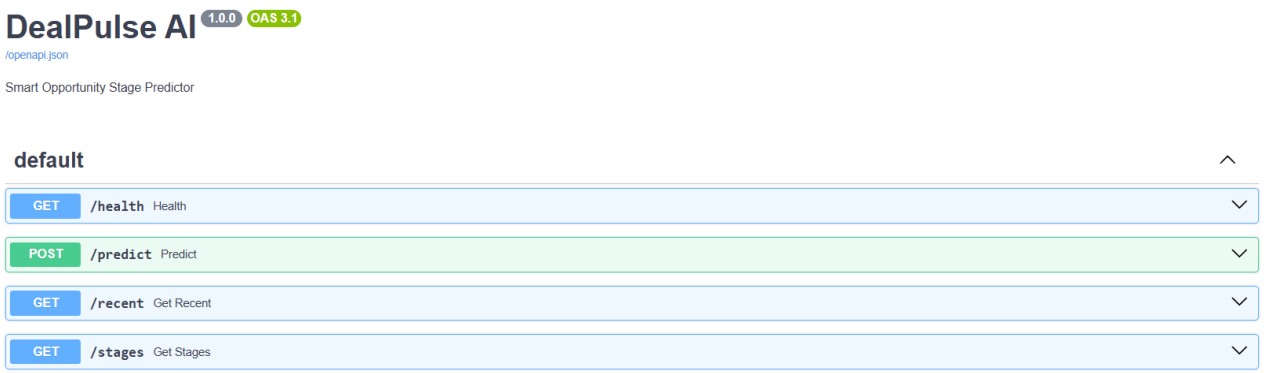
```python

@app.get("/recent")
def get_recent():
    return {"predictions": recent_predictions[-10:]}


@app.get("/stages")
def get_stages():
    return {
        "stages": [
            {"id": 0, "name": "Prospecting",  "color": "#6366f1"},
            {"id": 1, "name": "engaging",     "color": "#f59e0b"},
            {"id": 2, "name": "Closed Won",   "color": "#10b981"},
            {"id": 3, "name": "Closed Lost",  "color": "#ef4444"}
        ]
    }
```

Activity 3.5: Verify API via FastAPI Swagger Docs

FastAPI automatically generates interactive documentation at /docs. Start the server using:

uvicorn src.api.main:app –reload

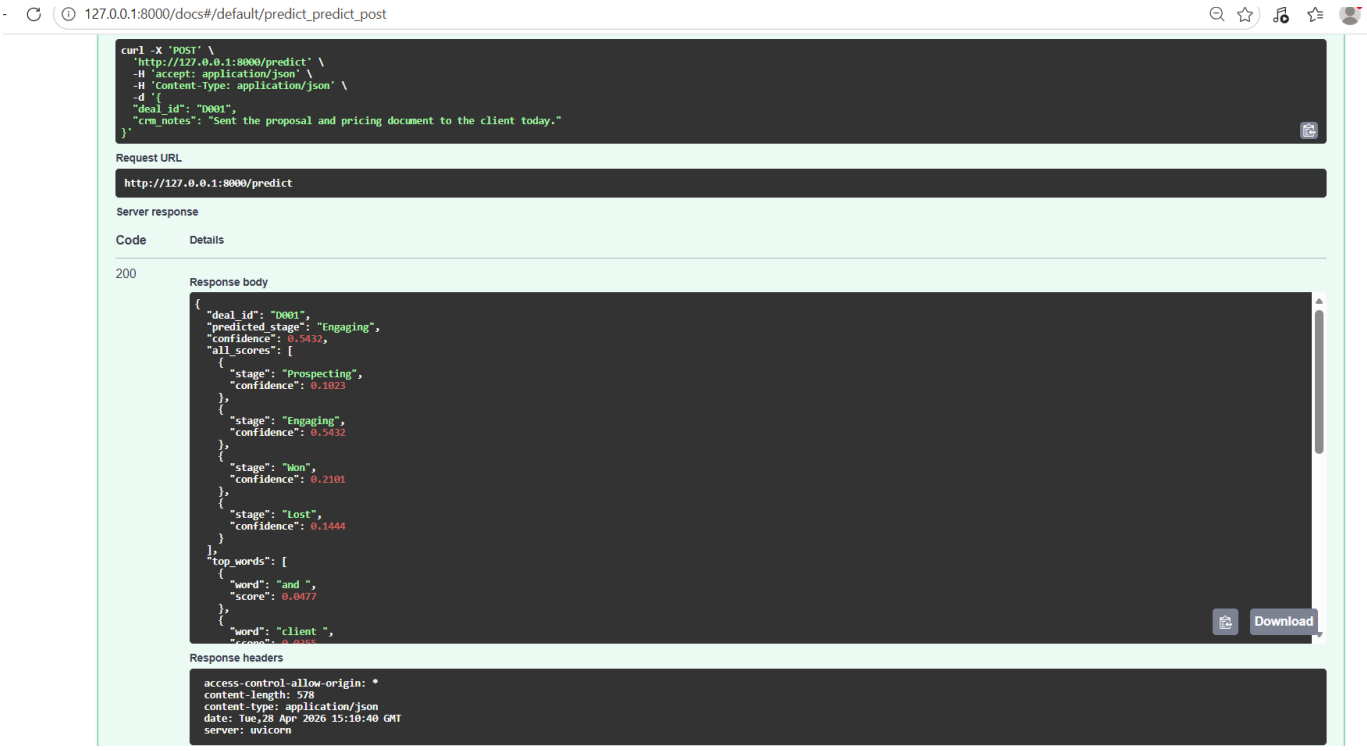
Open the browser at http://localhost:8000/docs to see the Swagger UI listing all four endpoints.



Expand the /predict endpoint, click "Try it out", and enter a sample input:

- deal_id: D001
- crm_notes: Sent the pricing proposal. Client asked for a discount on the annual plan.

Submit the request and verify the response contains predicted_stage, confidence, the full all_scores list, and top_words with SHAP scores.



MILESTONE 4: Feature Engineering & BERT Embeddings

This milestone focuses on transforming the cleaned and preprocessed CRM text data into rich numerical representations that the model can learn from. Using DistilBERT — a lightweight and efficient variant of the BERT transformer — each CRM note is converted into a 768-dimensional vector called a CLS token embedding. These embeddings capture deep semantic meaning from the sales conversation text, enabling the classifier to distinguish between deal stages like Prospecting, Negotiation, or Closed Won based on contextual language patterns. The pipeline covers tokenization, dataset construction, batched embedding extraction, and DataLoader preparation for downstream model training.

Activity 4.1: Load Tokenizer and Understand Tokenization

The first step in the embedding pipeline is loading the DistilBERT tokenizer from HuggingFace. This tokenizer converts raw CRM note text into token IDs and attention masks — the numerical format that transformer models understand.

The `tokenize_texts()` function then processes a list of CRM notes with a fixed `max_length` of 256 tokens, applying padding and truncation uniformly. Every input is padded to the same length so they can be batched efficiently during training.

```python


def tokenize_texts(
    texts: list,
    tokenizer,
    max_length: int = MAX_LENGTH
) -> dict:
    """
    Tokenize a list of texts.
    Returns dict with input_ids and attention_mask tensors.
    """
    encodings = tokenizer(
        texts,
        max_length=max_length,
        padding="max_length",
        truncation=True,
        return_tensors="pt"
    )
    print(f"[bert] Tokenized {len(texts)} texts → shape: {encodings['input_ids'].shape}")
    return encodings
```

Activity 4.2: Build Custom PyTorch Dataset and DataLoader

To integrate seamlessly with PyTorch's training loop, CRM note data is wrapped inside a custom CRMDataset class that inherits from torch.utils.data.Dataset. For each record, it tokenizes the text on-the-fly and returns a dictionary containing input_ids, attention_mask, and the corresponding integer label.

```python

class CRMDataset(Dataset):
    """
    PyTorch Dataset for CRM notes.
    Tokenizes text and returns input tensors for BERT.
    """
    def __init__(self, texts: list, labels: list, tokenizer, max_length: int = MAX_LENGTH):
        self.texts      = texts
        self.labels     = labels
        self.tokenizer  = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt"
        )
        return {
            "input_ids":      encoding["input_ids"].squeeze(0),
            "attention_mask": encoding["attention_mask"].squeeze(0),
            "label":          torch.tensor(self.labels[idx], dtype=torch.long)
        }
```

This dataset is then passed into a `DataLoader` using `build_dataloader()`, which handles batching and optional shuffling. Training data is shuffled to prevent order bias, while validation and test sets are kept in fixed order for consistent evaluation.

Activity 4.3: Extract CLS Token Embeddings

With the tokenizer and dataset ready, the `extract_embeddings()` function runs the preprocessed CRM notes through DistilBERT in batches without computing gradients. For each input, the model produces a sequence of hidden states and the first token — the `[CLS]` token — is extracted as the sentence-level representation. This 768-dimensional vector summarizes the entire CRM note semantically.

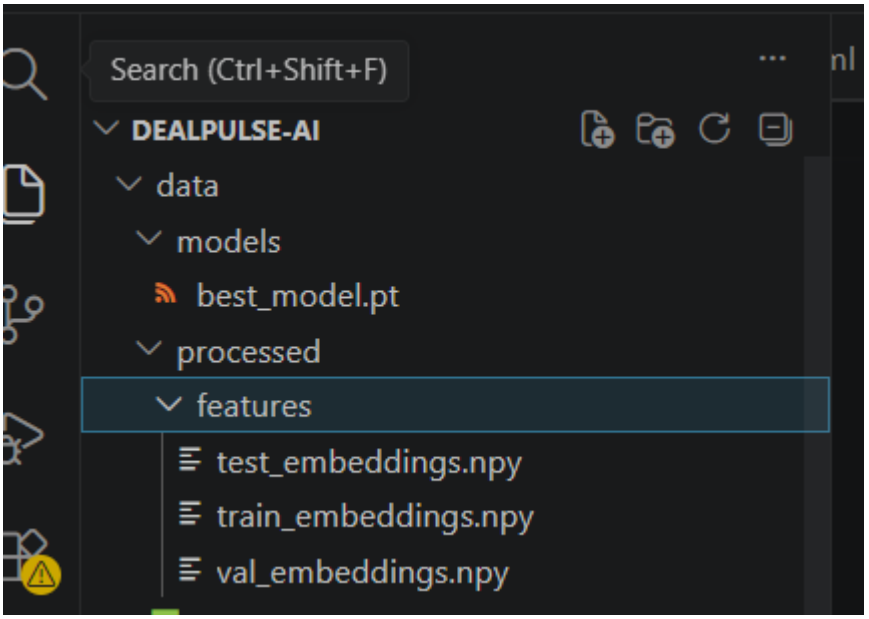
```python
) -> np.ndarray:

    for i in range(0, len(texts), batch_size):
        batch_texts = texts[i: i + batch_size]

        encodings = tokenizer(
            batch_texts,
            max_length=MAX_LENGTH,
            padding="max_length",
            truncation=True,
            return_tensors="pt"
        )

        input_ids      = encodings["input_ids"].to(device)
        attention_mask = encodings["attention_mask"].to(device)

        with torch.no_grad():
            outputs = model(input_ids=input_ids, attention_mask=attention_mask)

        # CLS token = first token = sentence-level representation
        cls_embeddings = outputs.last_hidden_state[:, 0, :]
        all_embeddings.append(cls_embeddings.cpu().numpy())

        print(f"[bert] Embedded batch {i // batch_size + 1} / {(len(texts) - 1) // batch_size + 1}")

    embeddings = np.vstack(all_embeddings)
    print(f"[bert] Final embedding shape: {embeddings.shape}")
```

The function processes inputs batch by batch and prints progress, making it easy to monitor long-running extraction jobs across large datasets.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    python

(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai> python src/features/bert_embedder.py
[bert] Embedded batch 69 / 370
[bert] Embedded batch 70 / 370
[bert] Embedded batch 71 / 370
[bert] Embedded batch 72 / 370
[bert] Embedded batch 73 / 370
[bert] Embedded batch 74 / 370
```

Activity 4.4: Save and Reload Embeddings

The embeddings for all three splits — train, validation, and test — are saved as `.npy` files under `data/processed/features/` using `save_embeddings()`. This makes them reusable without re-running the heavy transformer pass every time training is restarted.



After saving, the `load_embeddings()` function is used to verify that the files were written correctly. It reloads the `.npy` arrays and prints the shape, confirming expected dimensions before passing data to the model.

Activity 4.5: Verify Device Configuration

The embedding pipeline is device-aware — it automatically detects whether a CUDA GPU is available and moves both the model and tensors to the appropriate device. This ensures the same code runs efficiently on both local CPU machines and GPU-enabled environments like Google Colab or cloud instances.

```
[model] Loaded from data\models\best_model.pt
[model] Ready on cpu
[predictor] Ready!
INFO:     Started server process [24604]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

The successful completion of this milestone ensures that every CRM note from the sales pipeline dataset has been transformed into a rich semantic vector. These BERT embeddings capture nuanced language signals — the difference between "proposal sent to client" and "contract signed today" — that simple keyword-based features would miss entirely. With train, validation, and test embeddings saved and verified, the project is fully prepared for the model training phase in Milestone 5.

MILESTONE 5: Model Training & Experiment Tracking

This milestone covers the end-to-end training of the DealPulse AI classifier using the fine-tuned DistilBERT architecture. The model is trained on the preprocessed CRM note embeddings to predict deal stages from raw conversation text. Training is managed through a structured loop with per-epoch validation, best model checkpointing, and full MLflow experiment tracking to log hyperparameters, metrics, and model artifacts. The goal is to produce a robust, generalizable classifier that accurately maps CRM language to one of six deal stages.

Activity 5.1: Initialize Model, Optimizer, and Scheduler

Before training begins, the DealStageClassifier is initialized using load_model() from bert_classifier.py. The model loads DistilBERT as its backbone and attaches a custom classification head — a two-layer feedforward network with dropout and ReLU activation — that maps the 768-dimensional CLS embedding down to the number of target classes.

The AdamW optimizer is used with a learning rate of `2e-5`, which is standard for fine-tuning transformer models. A linear warmup scheduler is also initialized, warming up over the first 10% of total training steps before decaying, which helps stabilize early training.

```python

def train():
    train_df = pd.read_csv(PROCESSED_DIR / "train.csv")
    val_df   = pd.read_csv(PROCESSED_DIR / "val.csv")
    test_df  = pd.read_csv(PROCESSED_DIR / "test.csv")

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    train_loader, val_loader, test_loader = build_all_dataloaders(
        train_df, val_df, test_df, tokenizer, BATCH_SIZE
    )

    model, device = load_model()
    criterion     = nn.CrossEntropyLoss()
    optimizer     = AdamW(model.parameters(), lr=LR)
    total_steps   = len(train_loader) * EPOCHS
    scheduler     = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=total_steps // 10,
        num_training_steps=total_steps
    )

    MODEL_SAVE.parent.mkdir(parents=True, exist_ok=True)
    best_f1 = 0.0

```

Activity 5.2: Run the Training Loop

Training runs for 3 epochs using the `train_epoch()` function. In each epoch, the model processes every batch from the training DataLoader, computes CrossEntropy loss, backpropagates gradients, and updates weights via the optimizer and scheduler. Training accuracy and loss are tracked per epoch.

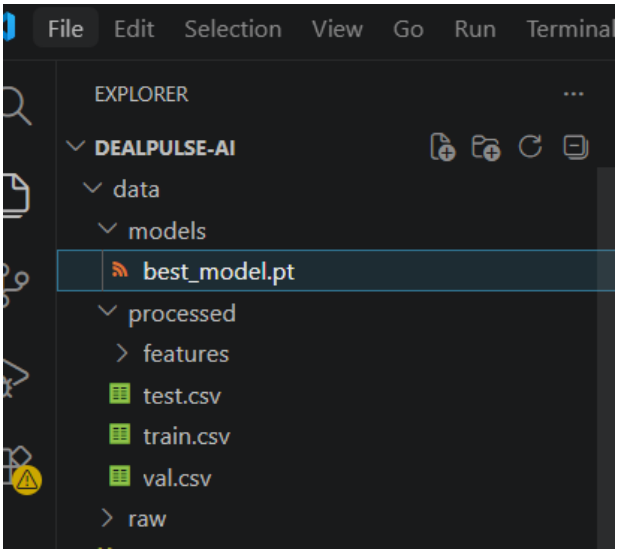
```python


def train_epoch(model, loader, optimizer, scheduler, criterion, device):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for batch in loader:
        input_ids      = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels         = batch["label"].to(device)
        optimizer.zero_grad()
        logits = model(input_ids, attention_mask)
        loss   = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        scheduler.step()
        total_loss += loss.item()
        correct    += (logits.argmax(1) == labels).sum().item()
        total      += len(labels)
    return total_loss / len(loader), correct / total


```

After each training epoch, `eval_epoch()` is called on the validation set without gradient computation. It collects all predictions and ground truth labels, then computes weighted F1 score and accuracy to assess generalization.

```
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai> python src/features/bert_embedder.py
[bert] Embedded batch 67 / 80
[bert] Embedded batch 68 / 80
[bert] Embedded batch 69 / 80
[bert] Embedded batch 70 / 80
[bert] Embedded batch 71 / 80
[bert] Embedded batch 72 / 80
[bert] Embedded batch 73 / 80
[bert] Embedded batch 74 / 80
[bert] Embedded batch 75 / 80
[bert] Embedded batch 76 / 80
[bert] Embedded batch 77 / 80
[bert] Embedded batch 78 / 80
[bert] Embedded batch 79 / 80
[bert] Embedded batch 80 / 80
[bert] Final embedding shape: (1268, 768)
[bert] Embeddings saved to data\processed\features\test_embeddings.npy

[bert] All embeddings extracted and saved successfully!
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai>
```

Activity 5.3: Save the Best Model Checkpoint

The training loop tracks the best validation F1 score seen so far. Whenever a new epoch achieves a higher F1 than the previous best, the model's state dictionary is saved to `data/models/best_model.pt` using `torch.save()`. This ensures the final saved model is the one that generalized best to unseen validation data, not simply the last epoch.



Activity 5.4: Track Experiments with MLflow

Every training run is logged inside an MLflow experiment called DealPulse-AI. At the start of the run, hyperparameters — epochs, lr, and batch_size — are logged using mlflow.log_params(). After each epoch, training loss, training accuracy, validation loss, validation accuracy, and validation F1 are logged as time-series metrics using mlflow.log_metrics() with the epoch number as the step.

src > models > train.py

```python
def train():
    with mlflow.start_run():
        mlflow.log_params({"epochs": EPOCHS, "lr": LR, "batch_size": BATCH_SIZE})

        for epoch in range(1, EPOCHS + 1):
            train_loss, train_acc = train_epoch(
                model, train_loader, optimizer, scheduler, criterion, device
            )
            val_loss, val_acc, val_f1 = eval_epoch(model, val_loader, criterion, device)

            print(f"Epoch {epoch}/{EPOCHS} | "
                  f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
                  f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f} | Val F1: {val_f1:.4f}")

            mlflow.log_metrics({
                "train_loss": train_loss,
                "train_acc":  train_acc,
                "val_loss":   val_loss,
                "val_acc":    val_acc,
                "val_f1":     val_f1
            }, step=epoch)

            if val_f1 > best_f1:
                best_f1 = val_f1
                torch.save(model.state_dict(), MODEL_SAVE)
                print(f"[train] Best model saved (F1: {best_f1:.4f})")

        _, test_acc, test_f1 = eval_epoch(model, test_loader, criterion, device)
        mlflow.log_metrics({"test_acc": test_acc, "test_f1": test_f1})
        mlflow.pytorch.log_model(model, "deal_stage_model")
        print(f"\n[train] Done! Test Acc: {test_acc:.4f} | Test F1: {test_f1:.4f}")
```

At the end of training, the final test set performance is evaluated and logged, and the entire trained model is saved as an MLflow artifact using mlflow.pytorch.log_model().

Activity 5.5: Evaluate on Test Set

Once training completes, the best saved model is evaluated on the held-out test set using `eval_epoch()`. This provides an unbiased measure of how well the model performs on CRM notes it has never seen during training or validation.

The combination of high validation and test F1 scores confirms that the model has learned meaningful deal-stage patterns from CRM language rather than overfitting to training data. With the best checkpoint saved and all metrics tracked in MLflow, the trained model is ready to be loaded by the predictor module for live inference through the API.

MILESTONE 6: Model Evaluation, Explainability & API Deployment

This final milestone brings together the complete DealPulse AI pipeline by validating model performance through rigorous evaluation metrics, interpreting predictions using SHAP-based explainability, and deploying the trained classifier as a production-ready FastAPI service with an interactive browser-based UI. The goal is to ensure the model is not just accurate but also interpretable and accessible — enabling sales managers to understand *why* a deal is predicted to be in a particular stage, not just *what* the prediction is. Each component built across the previous five milestones converges here into a fully functional, end-to-end intelligent sales tool.

Activity 6.1: Model Evaluation on Test Set

After training is complete in `train.py`, the best saved checkpoint is loaded and run against the held-out test split. The `compute_metrics()` function inside `metrics.py` handles all evaluation logic — it accepts the true labels and predicted labels, then computes accuracy, weighted F1 score, a full per-class classification report, and the confusion matrix. Results are printed to the terminal and saved to `data/processed/metrics.json` via `save_metrics()`.

```python
def compute_metrics(y_true: list, y_pred: list) -> dict:
    """Compute all evaluation metrics"""
    stage_names = load_stage_names()

    acc = accuracy_score(y_true, y_pred)
    f1  = f1_score(y_true, y_pred, average="weighted")
    cm  = confusion_matrix(y_true, y_pred)
    report = classification_report(y_true, y_pred, target_names=stage_names)

    print(f"\n{'='*40}")
    print(f"Accuracy : {acc:.4f}")
    print(f"F1 Score : {f1:.4f}")
    print(f"\nClassification Report:\n{report}")
    print(f"Confusion Matrix:\n{cm}")
    print(f"{'='*40}\n")

    return {
        "accuracy":  acc,
        "f1_score":  f1,
        "confusion_matrix": cm.tolist(),
        "report":    report
    }
```

The `save_metrics()` function filters out the human-readable report string and persists only the numeric values to JSON so they can be referenced later by the API or UI layer.

```python
    }


def save_metrics(metrics: dict, path: str = "data/processed/metrics.json"):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    metrics_to_save = {k: v for k, v in metrics.items() if k != "report"}
    with open(path, "w") as f:
        json.dump(metrics_to_save, f, indent=2)
    print(f"[metrics] Saved to {path}")
```

Activity 6.2: SHAP-Based Prediction Explainability

The `explainability.py` module uses SHAP's `Explainer` with a `Text` masker built from the DistilBERT tokenizer to compute token-level importance scores. This tells the model *which words* in a CRM note drove the prediction toward a particular deal stage — making the classifier transparent and trustworthy for business users.

```python
def explain_prediction(
    model,
    tokenizer,
    device,
    background_texts: list
) -> dict:
    """
    Use SHAP to explain which words drove the prediction.
    Returns top words and their importance scores.
    """
    stage_names = load_stage_names()

    # Wrap predict for SHAP
    def predictor(texts):
        return predict_proba(list(texts), model, tokenizer, device)

    # SHAP explainer
    explainer   = shap.Explainer(predictor, shap.maskers.Text(tokenizer))
    shap_values = explainer([text])

    # Get predicted class
    probs           = predict_proba([text], model, tokenizer, device)[0]
    predicted_id    = int(np.argmax(probs))
    predicted_stage = stage_names[predicted_id]

    # Top words for predicted class
    values = shap_values[0, :, predicted_id].values
    tokens = shap_values[0, :, predicted_id].data

    word_scores = sorted(
        zip(tokens, values),
        key=lambda x: abs(x[1]),
        reverse=True
```

The `predict_proba()` function is wrapped inside a local `predictor` closure and passed directly to SHAP, so the explainer treats the entire tokenizer-to-softmax pipeline as a black box and attributes importance scores at the word level. The top words and their scores are returned as a structured list of `{"word": ..., "score": ...}` dictionaries.

```python
if __name__ == "__main__":
    from models.bert_classifier import load_model      # ← CHANGED: removed 'src.'
    from transformers import AutoTokenizer

    model, device = load_model("data/models/best_model.pt")
    tokenizer     = AutoTokenizer.from_pretrained(MODEL_NAME)

    sample_text = "Sent the pricing proposal. Client asked for a discount on annual plan."
    background  = ["Follow up call scheduled", "Contract signed today", "Lost to competitor"]

    result = explain_prediction(sample_text, model, tokenizer, device, background)
```

Activity 6.3: API Schema & Predictor Setup

The `schemas.py` file defines the Pydantic models that govern all request and response contracts for the FastAPI service. `PredictRequest` enforces that every incoming call provides a `deal_id` string and a `crm_notes` string. `PredictResponse` structures the output to include the predicted stage name, confidence score, all six stage probabilities, and the top SHAP words.

```python
from pydantic import BaseModel, Field
from typing import List, Dict, Any


class PredictRequest(BaseModel):
    deal_id:   str = Field(..., example="D001")
    crm_notes: str = Field(..., example="Sent the proposal and pricing document to the client t


class StageConfidence(BaseModel):
    stage:      str
    confidence: float


class TopWord(BaseModel):          # ← ADDED: proper schema for top words
    word:  str
    score: float


class PredictResponse(BaseModel):
    deal_id:          str
    predicted_stage:  str
    confidence:       float
    all_scores:       List[StageConfidence]
    top_words:        List[TopWord]  # ← CHANGED: was List[Dict[str, float]]


class HealthResponse(BaseModel):
    status:  str
    model:   str
    version: str
```

The `predictor.py` file implements a `Predictor` singleton class that loads the model, tokenizer, and label names exactly once at startup and reuses them across all API requests. The `predict()` method calls `predict_proba()` to get softmax scores, then calls `explain_prediction()` with three hardcoded background texts to generate the SHAP explanation, and returns everything as a structured dictionary.

```python
class Predictor:
    """
    Singleton predictor — loads model once,
    reuses for every API request.
    """
    def __init__(self):
        self.stage_names = load_stage_names()
        self.model, self.device = load_model(str(MODEL_PATH))
        self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        print("[predictor] Ready!")
```

Activity 6.4: FastAPI Endpoint Setup

The `main.py` file wires together the entire serving layer. It creates the FastAPI app with CORS middleware enabled so the browser UI can communicate with it freely. The `/predict` endpoint accepts a `PredictRequest`, calls `predictor.predict()`, appends the result to the in-memory `recent_predictions` list capped at 50 entries, and returns a `PredictResponse`. Supporting endpoints include `/health` for status checks, `/recent` for the last 10 predictions, and `/stages` for the color-coded stage metadata used by the UI.

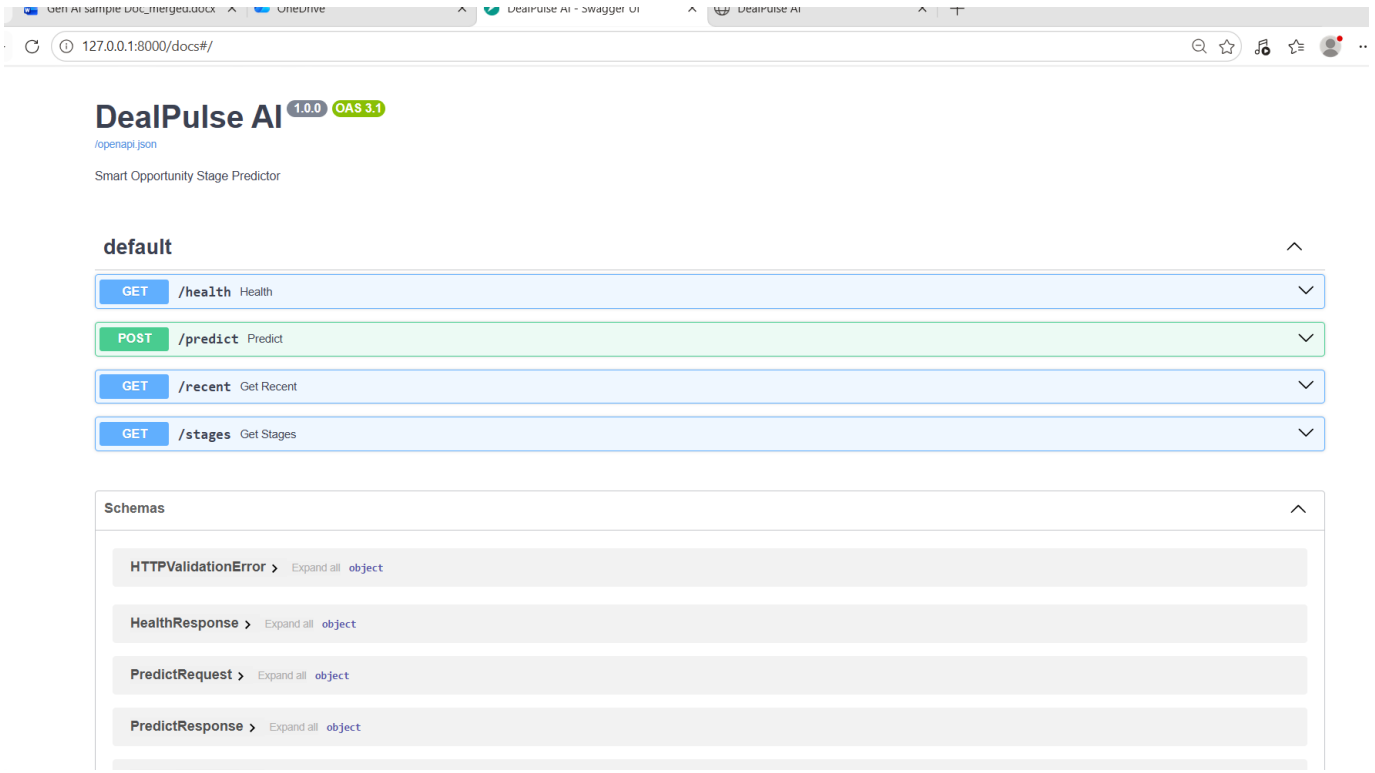
```python
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    try:
        result = predictor.predict(request.deal_id, request.crm_notes)

        # Save to recent predictions
        recent_predictions.append(result)
        if len(recent_predictions) > 50:
            recent_predictions.pop(0)

        return result

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

```
[bert] All embeddings extracted and saved successfully!
(venv) PS C:\Users\HP\OneDrive\Documents\dealpulse-ai> uvicorn src.api.main:app --reload --port 8000
INFO:     Will watch for changes in these directories: ['C:\\Users\\HP\\OneDrive\\Documents\\dealpulse-ai']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [22316] using StatReload
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
d faster downloads.
Loading weights: 100%|                                                    | 100/100 [00:00<00:00, 5894.77it/s]
[transformers] DistilBertModel LOAD REPORT from: distilbert-base-uncased
```

```
[model] Loaded from data\models\best_model.pt
[model] Ready on cpu
[predictor] Ready!
INFO:     Started server process [24604]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Activity 6.5: API Testing via Swagger UI

FastAPI automatically generates an interactive Swagger documentation page at `/docs`. This allows direct testing of all endpoints without any external tool. The `/predict` endpoint can be tested by pasting a sample `deal_id` and `crm_notes` value into the request body and executing it — the response shows the predicted stage, confidence, all stage scores, and the top SHAP words.



127.0.0.1:8000/docs#/default/predict_predict_post

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "deal_id": "D001",
  "crm_notes": "Sent the proposal and pricing document to the client today."
}'
```

Request URL

```
http://127.0.0.1:8000/predict
```

Server response

Code	Details
200	Response body


```
{
  "deal_id": "D001",
  "predicted_stage": "Engaging",
  "confidence": 0.5432,
  "all_scores": [
    {
      "stage": "Prospecting",
      "confidence": 0.1023
    },
    {
      "stage": "Engaging",
      "confidence": 0.5432
    },
    {
      "stage": "Won",
      "confidence": 0.2101
    },
    {
      "stage": "Lost",
      "confidence": 0.1444
    }
  ],
  "top_words": [
    {
      "word": "and ",
      "score": 0.0477
    },
    {
      "word": "client ",
```

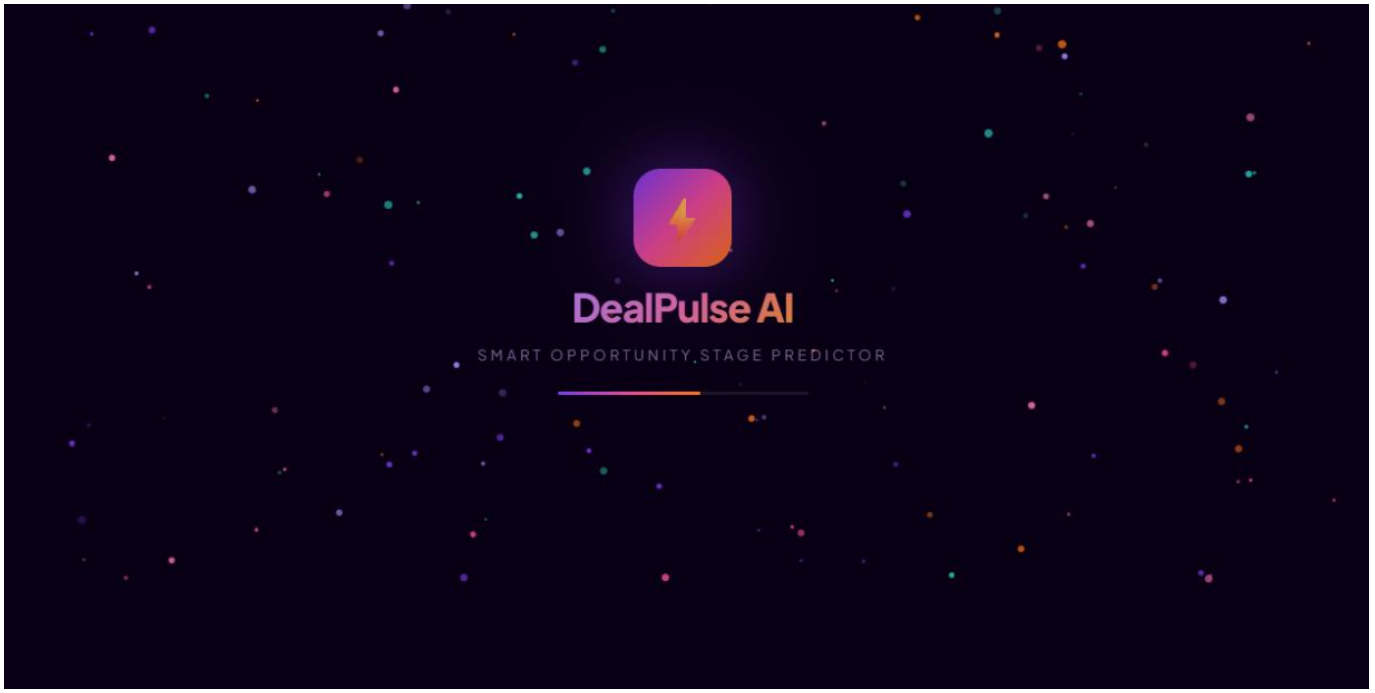

Download

Response headers

```
access-control-allow-origin: *
content-length: 578
content-type: application/json
date: Tue,28 Apr 2026 15:10:40 GMT
server: uvicorn
```

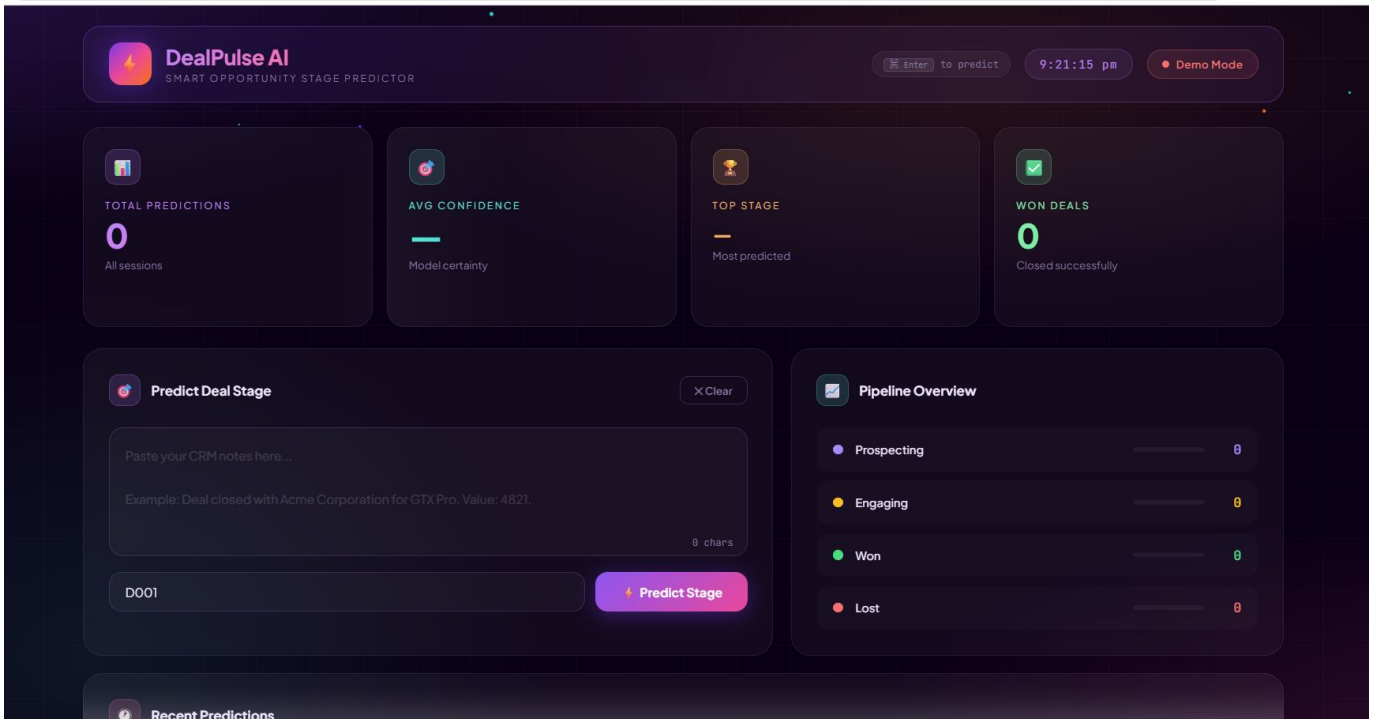
Activity 6.6: Browser UI

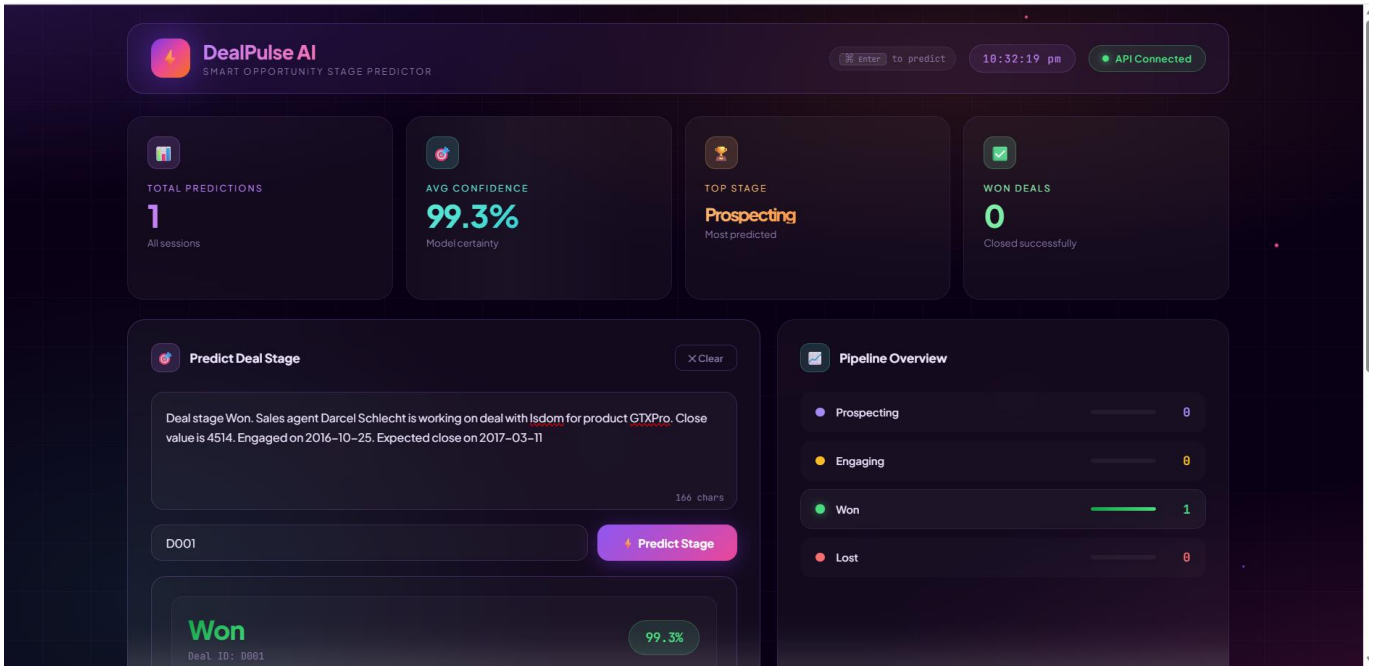
The `ui/index.html` file provides a standalone single-page interface for interacting with the DealPulse AI API without using Swagger. Users enter a Deal ID and paste CRM notes into a text area, then click Predict. The UI calls the `/predict` endpoint, displays the predicted stage name with its confidence score styled in the stage's brand color, renders a horizontal bar chart of all six stage probabilities, and shows a ranked word importance table of the top SHAP tokens.

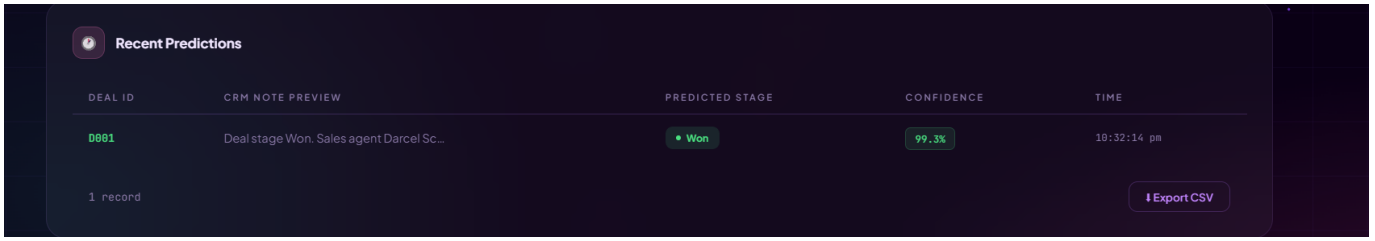


DealPulse AI

SMART OPPORTUNITY STAGE PREDICTOR







Conclusion

DealPulse AI demonstrates a complete, production-grade machine learning pipeline applied to a real-world sales intelligence problem. Starting from raw CRM data, the system ingests and preprocesses deal records, engineers contextual embeddings using DistilBERT, fine-tunes a multi-class classifier across six deal stages, tracks experiments with MLflow, explains predictions with SHAP, and serves results through a FastAPI backend with a browser UI.

The combination of transformer-based NLP and explainable AI makes DealPulse AI not just a predictor but a decision-support tool — one that gives sales teams both the forecast and the reasoning behind it. The modular architecture ensures each component is independently testable and extensible, making it straightforward to retrain on updated data, add new stages, or integrate with a live CRM platform in production.